FUTURE INTERNS CYBER SECURITY INTERNSHIP

VULNERABILITY ASSESSMENT REPORT

Altoro Mutual — demo.testfire.net

TASK 1 | FUTURE_CS_01

INTERN NAME :LIZO NDAWANA

CIN ID: FIT/MAR26/CS7183

DON’T LET HACKERS IN!

1.Executive Summary

A passive vulnerability assessment was conducted on the Altoro Mutual demo banking web application hosted at http://demo.testfire.net (IP: 65.61.137.117), running on Apache Tomcat/Coyote JSP engine 1.1 on an Amazon Web Services EC2 instance in the us-west-2 region.

The assessment was conducted using three tools: Nmap 7.99 for network port and service discovery, OWASP ZAP 2.17.0 for automated web vulnerability scanning, and Browser DevTools for manual cookie and header inspection. No active exploitation was performed — all testing was conducted using passive and non-destructive techniques.

A total of 13 security findings were identified across four risk categories. The most critical finding is a confirmed SQL Injection vulnerability in the jobs search parameter, which could allow an attacker to manipulate the underlying database. Additional issues include missing security headers, insecure cookie configuration, and server version disclosure.

Overall, the application presents a HIGH-RISK security posture. Immediate remediation is recommended for the SQL Injection finding, followed by addressing the medium-risk configuration issues.

Risk Summary

Risk Level	Count	% of Total	Status
High	**1**	7.70%	Open
Medium	**5**	38.50%	Open
Low	**5**	38.50%	Open
Informational	**2**	15.40%	Open

2. Scope & Tools Used

2.1 Assessment Scope

This assessment was limited to passive and non-destructive scanning of the target web application. No active exploitation, credential brute-forcing, or destructive testing was performed. The following target was in scope:

- Target URL: http://demo.testfire.net
- Target IP: 65.61.137.117
- Hosting: Amazon Web Services EC2 (us-west-2 region)
- Application: Altoro Mutual — Intentionally vulnerable banking demo
- Assessment Type: Passive vulnerability assessment only

2.2 Tools Used

Tool	Version	Purpose	Findings Contributed
Nmap	7.99	Network port scanning, service and version detection, SSL certificate analysis	Open ports, server version, SSL cert details, AWS infrastructure
OWASP ZAP	2.17.0	Automated passive web vulnerability scanning, spider	11 alerts including SQL Injection, missing headers, cookie issues
Browser DevTools	Chrome	Manual cookie inspection, HTTP header review, network	JSESSIONID cookie missing Secure/HttpOnly/Same

3. Nmap Scan Results

An Nmap scan was conducted using the following command:
nmap -sV -sC -Pn demo.testfire.net

Scan completed in 73.26 seconds. Host confirmed up at IP 65.61.137.117, hosted on AWS EC2 (us-west-2). Results are summarised below:

Port	State	Service	Version	Security Note
80/tcp	**Open**	HTTP	Apache Tomcat/Coyote JSP engine 1.1	Unencrypted HTTP — version disclosed
443/tcp	**Open**	HTTPS (SSL)	Apache Tomcat/Coyote JSP engine 1.1	SSL cert valid until 21 Jun 2026
8080/tcp	**Open**	HTTP	Apache Tomcat/Coyote JSP engine 1.1	Extra unencrypted port — unnecessary exposure
8443/tcp	**Closed**	HTTPS-alt	N/A	Closed — no issue
996 others	**Filtered**	Various	N/A	Blocked by firewall — good practice

4. Vulnerability Findings Summary

The following table summarises all 13 vulnerabilities identified during this assessment across all three tools used.

#	Vulnerability	Source	Risk Level	Affected URL / Component
1	**SQL Injection - SQLite (Time Based)**	OWASP ZAP	**High**	http://demo.testfire.net/index.jsp?content=inside_jobs.htm&job=Teller:ConsumerBanking
2	**Absence of Anti-CSRF Tokens**	OWASP ZAP	**Medium**	Multiple pages: /index.jsp, /login.jsp, /subscribe.jsp
3	**Content Security Policy (CSP) Header Not Set**	OWASP ZAP	**Medium**	Multiple pages across demo.testfire.net
4	**Missing Anti-Clickjacking Header (X-Frame-Options)**	OWASP ZAP	**Medium**	Multiple pages across demo.testfire.net
5	**Sub Resource Integrity Attribute Missing**	OWASP ZAP	**Medium**	http://demo.testfire.net/index.jsp?content=personal_investments.htm
6	**Insecure Session Cookie (JSESSIONID)**	Browser DevTools	**Medium**	http://demo.testfire.net/ — JSESSIONID cookie
7	**Cookie without SameSite Attribute**	OWASP ZAP	**Low**	http://demo.testfire.net — JSESSIONID cookie
8	**Cross-Domain JavaScript Source File Inclusion**	OWASP ZAP	**Low**	http://demo.testfire.net
9	**Server Version Disclosure via HTTP Header**	Nmap + OWASP ZAP	**Low**	All responses — Server: Apache-Coyote/1.1
10	**X-Content-Type-Options Header Missing**	OWASP ZAP	**Low**	Multiple pages across demo.testfire.net
11	**Unencrypted HTTP Traffic (No HTTPS Enforcement)**	Nmap + Browser	**Low**	http://demo.testfire.net — port 80 open
12	**Information Disclosure - Suspicious**	OWASP ZAP	**Informational**	Multiple pages — HTML source code comments
13	**Session Management Response Identified**	OWASP ZAP	**Informational**	http://demo.testfire.net — session token in response

5. Detailed Findings

Finding 1 — SQL Injection (SQLite Time-Based)

Risk Level	**HIGH**
Tool Detected	OWASP ZAP 2.17.0
Affected URL	http://demo.testfire.net/index.jsp?content=inside_jobs.htm&job=Teller:ConsumerBanking
Parameter	job
CWE / OWASP	CWE-89 | OWASP Top 10 2021: A03 Injection | PCI DSS | HIPAA

Description:
ZAP detected a time-based SQL Injection vulnerability in the 'job' GET parameter on the jobs listing page. The scanner injected SQLite timing payloads into the parameter and measured response delays. When the payload 'case randomblob(1000000000) when not null then 1 else 1 end' was injected, the server took 530 milliseconds to respond, demonstrating that the database query execution time is directly controllable by user input.

Business Impact:
•An attacker could extract the entire database contents including customer account numbers, balances, and personal information
•Authentication could be bypassed to gain admin access without valid credentials
•For a real banking application, this would represent a critical data breach risk with regulatory and financial consequences

Remediation:
•Replace dynamic SQL queries with parameterised queries or prepared statements
•Implement server-side input validation and sanitisation on all user-supplied parameters
•Apply the principle of least privilege to database accounts — the web app should not have DROP or DELETE permissions
•Enable a Web Application Firewall (WAF) to detect and block SQL injection attempts

Finding 2 — Absence of Anti-CSRF Tokens

Risk Level	**MEDIUM**
Affected Pages	/index.jsp, /login.jsp, /subscribe.jsp

Description:
Forms across the application do not include CSRF tokens. This means a malicious website could trick a logged-in user's browser into submitting unauthorised requests to the banking application — such as transferring funds or changing account settings — without the user's knowledge.

Remediation:
- Implement CSRF tokens on all state-changing forms
- Validate the token server-side on every POST request
- Use the SameSite=Strict cookie attribute as an additional defense layer

Finding 3 — Content Security Policy (CSP) Header Not Set

Risk Level	**MEDIUM**
Affected Pages	5 pages detected — site-wide issue

Description:
The Content-Security-Policy HTTP response header is absent across the application. CSP is a browser security mechanism that restricts which resources (scripts, styles, images) a page can load. Without it, the application is significantly more vulnerable to Cross-Site Scripting (XSS) attacks and data injection attacks.

Remediation:
- Add a Content-Security-Policy header in the web server configuration
- Start with a restrictive policy such as: Content-Security-Policy: default-src 'self'

Gradually expand the policy to allow only explicitly trusted sources

Finding 4 — Missing Anti-Clickjacking Header (X-Frame-Options)

Risk Level	**MEDIUM**

Your paragraph teDescription:
The X-Frame-Options header is missing from HTTP responses. This allows the application to be embedded inside an iframe on a malicious website. Attackers can use this technique — known as clickjacking — to trick users into clicking on hidden buttons that perform banking actions (such as approving a transfer) while they think they are interacting with a different website.

Remediation:
•Add the response header: X-Frame-Options: DENY
•Alternatively use CSP with frame-ancestors directive: Content-Security-Policy: frame-ancestors 'none'

Finding 5 — Insecure Session Cookie (JSESSIONID)

Risk Level	**MEDIUM**
Cookie Name	JSESSIONID
Missing Flags	**Secure flag, HttpOnly flag, SameSite attribute — all missing**
Detected By	Browser DevTools — Application > Cookies > http://demo.testfire.net

Description:
Manual inspection using Browser DevTools confirmed that the JSESSIONID session cookie is set without three critical security flags. Without the Secure flag, the session token can be transmitted over unencrypted HTTP. Without HttpOnly, JavaScript code can access and steal the cookie. Without SameSite, the cookie is vulnerable to CSRF attacks. A stolen session token gives an attacker full access to the user's banking session.

Remediation:
•Set the Secure flag to ensure the cookie is only sent over HTTPS connections
•Set the HttpOnly flag to prevent JavaScript from accessing the session cookie
•Set SameSite=Strict or SameSite=Lax to prevent cross-site request forgery
•In Apache Tomcat, configure these in the web.xml or context.xml session configuration

Finding 6 — Server Version Disclosure

Risk Level	**LOW**
Disclosed Info	Server: Apache-Coyote/1.1 (confirmed by both Nmap and ZAP HTTP headers)

Description:
Both the Nmap scan and ZAP HTTP response analysis confirm that every server response includes a Server header revealing the exact web server software and version: Apache-Coyote/1.1. This information assists attackers in identifying known vulnerabilities specific to that version and planning targeted attacks.

Remediation:
•In Apache Tomcat, set 'server' attribute in server.xml to a blank value or generic string
•Remove or obfuscate the Server HTTP response header

Finding 7 — Unencrypted HTTP Traffic

Risk Level: LOW
Source: Nmap + Browser DevTools
Description:
Port 80 is open and the site loads over plain HTTP with no automatic redirect to HTTPS, confirmed by the 'Not Secure' warning in the browser address bar and Nmap detecting port 80 as open. Any data transmitted — including login credentials — can be intercepted by an attacker on the same network using a tool like Wireshark.

Remediation:
•Force all traffic to HTTPS using a 301 redirect from port 80 to 443
•Implement HTTP Strict Transport Security (HSTS) header

Finding 8 — Sub Resource Integrity (SRI) Attribute Missing

Risk Level: MEDIUM
Affected URL: http://demo.testfire.net/index.jsp?content=personal_investments.htm
Description:
A script or stylesheet is loaded from an external server without an integrity attribute. If an attacker compromises the external server, they could inject malicious code into the script file which would then execute in users' browsers.

Remediation:
•Add integrity and crossorigin attributes to all externally-sourced script and link tags
•Generate SRI hashes using https://www.srihash.org/

Findings 9–13 — Additional Low & Informational Issues

Finding	Risk	Remediation
Cookie without SameSite Attribute	**Low**	Set SameSite=Strict on all session cookies to prevent cross-site request forgery attacks
Cross-Domain JavaScript Inclusion	**Low**	Review all externally loaded scripts and add SRI hashes; remove unnecessary third-party scripts
X-Content-Type-Options Header Missing	**Low**	Add response header: X-Content-Type-Options: nosniff to prevent MIME-type sniffing attacks
Information Disclosure - Suspicious Comments	**Informational**	Remove all developer comments from production HTML source code before deployment
Session Management Response Identified	**Informational**	Ensure session tokens are sufficiently random, expire after inactivity, and are invalidated on logout

6. Conclusion & Recommendations

The vulnerability assessment of the Altoro Mutual web application (demo.testfire.net) revealed a total of 13 security findings across four risk categories. The application demonstrates a HIGH-RISK security posture with a confirmed SQL Injection vulnerability that, in a real-world banking environment, could lead to complete database compromise, regulatory penalties, and significant financial and reputational damage.

The majority of medium-risk findings relate to missing security headers and insecure cookie configuration — issues that are straightforward to remediate through server configuration changes and require no code changes. The single high-risk SQL Injection finding requires code-level fixes to implement parameterised queries.

Priority Remediation Roadmap

Priority	Risk	Action	
1	High	Fix SQL Injection — implement	Immediate — within 24 hours
2	Medium	Add security headers (CSP, X-Frame-Options)	Short-term — within 1 week
3	Medium	Implement CSRF tokens on all forms	Short-term — within 1 week
4	Medium	Secure JSESSIONID cookie (Secure, HttpOnly, SameSite)	Short-term — within 1 week
5	Low	Enforce HTTPS, remove HTTP access on port 80	Medium-term — within 1 month
6	Low	Remove server version disclosure from HTTP headers	Medium-term — within 1 month

This assessment was conducted as part of the Future Interns Cyber Security Internship (Task 1 — FUTURE_CS_01). All findings were identified using passive, non-destructive techniques on an intentionally vulnerable demo environment provided for educational purposes.Supporting evidence is available in the linked GitHub repository.